Basic Concepts

Record

The smallest unit that you can load from and store in the database. Records come in four types:

- Documents
- Blobs
- Vertices
- Edges

A **Record** is the smallest unit that can be loaded from and stored into the database. A record can be a Document, a Blob a Vertex or even an Edge.

Document

The Document is the most flexible record type available in OrientDB. Documents are softly typed and are defined by schema classes with defined constraints, but you can also use them in a schema-less mode too.

Documents handle fields in a flexible manner. You can easily import and export them in JSON format. For example,

```
{
     "name"      : "Jay",
     "surname"   : "Miner",
     "job"       : "Developer",
     "creations" : [
          {
               "name"    : "Amiga 1000",
               "company" : "Commodore Inc."
          }, {
               "name"    : "Amiga 500",
               "company" : "Commodore Inc."
          }
     ]
}
```

For Documents, OrientDB also supports complex relationships. From the perspective of developers, this can be understood as a persistent `Map<String,Object>`.

BLOB

In addition to the Document record type, OrientDB can also load and store binary data. The BLOB record type was called `RecordBytes` before OrientDB v2.2.

Vertex

In Graph databases, the most basic unit of data is the node, which in OrientDB is called a vertex. The Vertex stores information for the database. There is a separate record type called the Edge that connects one vertex to another.

Vertices are also documents. This means they can contain embedded records and arbitrary properties.

Edge

In Graph databases, an arc is the connection between two nodes, which in OrientDB is called an edge. Edges are bidirectional and can only connect two vertices.

Edges can be regular or lightweight. The Regular Edge saves as a Document, while the Lightweight Edge does not. For an understanding of the differences between these, see Lightweight Edges.

For more information on connecting vertices in general, see Relationships, below.

Record ID

When OrientDB generates a record, it auto-assigns a unique unit identifier, called a Record ID, or RID. The syntax for the Record ID is the pound sign with the cluster identifier and the position. The format is like this:

`#<cluster>:<position>`.

- **Cluster Identifier**: This number indicates the cluster to which the record belongs. Positive numbers in the cluster identifier indicate persistent records. Negative numbers indicate temporary records, such as those that appear in result-sets for queries that use projections.
- **Position**: This number defines the absolute position of the record in the cluster.

> **NOTE**: The prefix character `#` is mandatory to recognize a Record ID.

Records never lose their identifiers unless they are deleted. When deleted, OrientDB

never recycles identifiers. Additionally, you can access records directly through their Record ID's. For this reason, you don't need to create a field to serve as the primary key, as you do in Relational databases.

Record Version

Records maintain their own version number, which increments on each update. In optimistic transactions, OrientDB checks the version in order to avoid conflicts at commit time.

Class

The concept of the Class is taken from the Object Oriented Programming paradigm. In OrientDB, classes define records. It is closest to the concept of a table in Relational databases.

Classes can be schema-less, schema-full or a mix. They can inherit from other classes, creating a tree of classes. Inheritance, in this context, means that a sub-class extends a parent class, inheriting all of its attributes.

Each class has its own clusters (data files). A non-abstract class (see below) must have at least one cluster defined, which functions as its default cluster. But, a class can support multiple clusters. When you execute a query against a class, it automatically propagates to all clusters that are part of the class. When you create a new record, OrientDB selects the cluster to store it in using a configurable strategy.

When you create a new class, by default, OrientDB creates new persistent clusters with the same name as the class, in lowercase, suffixed with underscore and an integer. As a default, OrientDB creates as many clusters per class as many cores (processors) the host machine has.

Eg. for class `Person`, OrientDB will create clusters `person`, `person_1`, `person_2` and so on so forth.

Abstract Class

The concept of an Abstract Class is one familiar to Object-Oriented programming. In OrientDB, this feature has been available since version 1.2.0. Abstract classes are classes used as the foundation for defining other classes. They are also classes that cannot have instances. For more information on how to create an abstract class, see CREATE CLASS.

This concept is essential to Object Orientation, without the typical spamming of the database with always empty, auto-created clusters.

> For more information on Abstract Class as a concept, see Abstract Type and Abstract Methods and Classes

Class vs. Cluster in Queries

The combination of classes and clusters is very powerful and has a number of use cases. Consider an example where you create a class `Invoice`, with two clusters `invoice2015` and `invoice2016`. You can query all invoices using the class as a target with `SELECT`.

```
orientdb>
SELECT FROM Invoice
```

In addition to this, you can filter the result-set by year. The class `Invoice` includes a `year` field, you can filter it through the `WHERE` clause.

```
orientdb>
SELECT FROM Invoice WHERE year = 2012
```

You can also query specific objects from a single cluster. By splitting the class `Invoice` across multiple clusters, (that is, one per year), you can optimize the query by narrowing the potential result-set.

```
orientdb>
SELECT FROM CLUSTER:invoice2012
```

Due to the optimization, this query runs significantly faster, because OrientDB can narrow the search to the targeted cluster.

Cluster

Where classes provide you with a logical framework for organizing data, clusters provide physical or in-memory space in which OrientDB actually stores the data. It is comparable to the collection in Document databases and the table in Relational databases.

When you create a new class, the `CREATE CLASS` process also creates physical clusters

that serve as the default location in which to store data for that class. OrientDB forms the cluster names using the class name, with all lower case letters. Beginning with version 2.2, OrientDB creates additional clusters for each class, (one for each CPU core on the server), to improve performance of parallelism.

> For more information, see the Clusters Tutorial.

Materialized View

A materialized view is a persistent object that contains the result of a query. In terms of SQL querying, it can be considered as the equivalent of a class, that means that it can be used as a target for queries.

A materialized view can be configured to be read-only or updatable. Updating a record of a materialized view results in the update of the original record (ie. the record from which the view raw was created).

Views can have indexes, like normal classes.

Relationships

OrientDB supports two kinds of relationships: **referenced** and **embedded**. It can manage relationships in a schema-full or schema-less scenario.

Referenced Relationships

In Relational databases, tables are linked through `JOIN` commands, which can prove costly on computing resources. OrientDB manges relationships natively without computing `JOIN`'s. Instead, it stores direct links to the target objects of the relationship. This boosts the load speed for the entire graph of connected objects, such as in Graph and Object database systems.

For example

```
                customer
  Record A     ------------->    Record B
CLASS=Invoice                 CLASS=Customer
  RID=5:23                       RID=10:2
```

Here, record `A` contains the reference to record `B` in the property `customer`. Note that both records are reachable by other records, given that they have a Record ID.

With the Graph API, Edges are represented with two links stored on both vertices to handle the bidirectional relationship.

1:1 and 1:*n* Referenced Relationships

OrientDB expresses relationships of these kinds using links of the `LINK` type.

1:*n* and *n*:*n* Referenced Relationships

OrientDB expresses relationships of these kinds using a collection of links, such as:

- `LINKLIST` An ordered list of links.
- `LINKSET` An unordered set of links, which does not accept duplicates.
- `LINKMAP` An ordered map of links, with `String` as the key type. Duplicates keys are not accepted.

With the Graph API, Edges connect only two vertices. This means that 1:*n* relationships are not allowed. To specify a 1:*n* relationship with graphs, create multiple edges.

Embedded Relationships

When using Embedded relationships, OrientDB stores the relationship within the record that embeds it. These relationships are stronger than Reference relationships. You can represent it as a UML Composition relationship.

Embedded records do not have their own Record ID, given that you can't directly reference it through other records. It is only accessible through the container record.

In the event that you delete the container record, the embedded record is also deleted. For example,

```
                  address
  Record A     <>---------->   Record B
CLASS=Account               CLASS=Address
  RID=5:23                     NO RID!
```

Here, record `A` contains the entirety of record `B` in the property `address`. You can reach record `B` only by traversing the container record. For example,

```
orientdb>
SELECT FROM Account WHERE address.city = 'Rome'
```

1:1 and *n*:1 Embedded Relationships

OrientDB expresses relationships of these kinds using the `EMBEDDED` type.

1:*n* and *n*:*n* Embedded Relationships

OrientDB expresses relationships of these kinds using a collection of links, such as:

- `EMBEDDEDLIST` An ordered list of records.
- `EMBEDDEDSET` An unordered set of records, that doesn't accept duplicates.
- `EMBEDDEDMAP` An ordered map of records as the value and a string as the key, it doesn't accept duplicate keys.

Inverse Relationships

In OrientDB, all Edges in the Graph model are bidirectional. This differs from the Document model, where relationships are always unidirectional, requiring the developer to maintain data integrity. In addition, OrientDB automatically maintains the consistency of all bidirectional relationships.

Database

The database is an interface to access the real Storage. IT understands high-level concepts such as queries, schemas, metadata, indices and so on. OrientDB also provides multiple database types. For more information on these types, see Database Types.

Each server or Java VM can handle multiple database instances, but the database name must be unique. You can't manage two databases at the same time, even if they are in different directories. To handle this case, use the `$` dollar character as a separator instead of the `/` slash character. OrientDB binds the entire name, so it becomes unique, but at the file system level it converts `$` with `/`, allowing multiple databases with the same name in different paths. For example,

```
test$customers -> test/customers
production$customers = production/customers
```

Database URL

OrientDB uses its own URL format, of engine and database name as `<engine>:<db-name>`.

Engine	Description	Example
plocal	This engine writes to the file system to store data. There is a LOG of changes to restore the storage in case of a crash.	`plocal:/temp/databases/petshop/petshop`
memory	Open a database completely in memory	`memory:petshop`
remote	The storage will be opened via a remote network connection. It requires an OrientDB Server up and running. In this mode, the database is shared among multiple clients. Syntax: `remote:<server>:[<port>]/db-name`. The port is optional and defaults to 2424.	`remote:localhost/petshop`

Database Usage

You must always close the database once you finish working on it.

> **NOTE**: OrientDB automatically closes all opened databases, when the process dies gracefully (not by killing it by force). This is assured if the Operating System allows a graceful shutdown.